SLO/SLI 계측 방식 기술 문서 v4.2

(Measurement Methods Spec v4.2 — `schemaVersion="slo.v3"` 유지, `config.format="v4.2"` 권장)

0) 빠른 개요(orientation)

- 이 문서는 "계측 방식"을 **Trigger(경계 생성) / Observe(스냅샷 수집) / Evaluate(엔진 평가)** 3레이어로 재정의하고, 그 조합을 **3가지 시나리오 + 1 가지 추가**로 제공한다.
- 현재 v3/v4에서 구현된 기본 경로는 **No Trigger + External Observe**이며, "외부 non-trigger는 의미가 없다"는 말은 **경계(시작/끝)가 전혀 없을 때**에 해당한다.

  지금 방식은 **세션이 경계를 잡으므로(=계측자 경계)** 충분히 의미가 있다.
- v4.2에서 핵심 확장은 **InsideAnnotation의 종료 정의(EndPolicy)를 표준화/옵션화**하는 것이다.

  (예: `StatusCondition`, `MetricDelta`, `Timeout`)
- Prometheus/OpenMetrics 텍스트 기반 스크랩이 기본이지만, 장기적으로 Observe 레이어를 플러그인화하여 **OTel(OTLP) 입력도 수용 가능한 방향**으로 설계를 고정한다.

1) 용어 및 모델(오늘 추가된 핵심)

1.1 3-레이어 모델

- **Trigger(경계 생성)**: 측정 윈도우의 시작/끝을 무엇으로 정의하는가?
  - `NoTrigger`(계측자 경계): 테스트/세션이 `Start/End`를 기록
  - `AnnotationTrigger`(시스템 경계): 특정 CR에 annotation을 남겨 시작을 정의하고, 종료는 EndPolicy로 정의
- **Observe(스냅샷 수집)**: 시작/끝 시점에 어떤 방법으로 입력 스냅샷(메트릭 맵)을 수집하는가?
  - `Inside/CurlPod`(클러스터 내부에서 curl pod로 scrape)
  - `Outside/Endpoint`(클러스터 밖에서 접근 가능한 endpoint로 scrape)
- **Evaluate(엔진)**: start/end 스냅샷으로 SLI 계산/판정/summary 생성

1.2 Write(산출물) 레이어(보조 레이어)

- Evaluate 결과를 **어디에/어떻게 기록할지**는 Write 레이어에서 결정한다.
- `ArtifactsDir==""` 면 write는 noop(계산은 가능).
- v4.2 권장: `NextSummaryPath` 같은 충돌 회피 규칙 유지.

1.3 canonicalize(정규화)

- `Canonicalize` 는 "동일 의미의 메트릭 키가 **항상 동일한 문자열 키**로 매칭되도록" 라벨 정렬/이스케이프 등을 적용하는 정규화이다.
- 목적: start/end 스냅샷 비교, 맵 키 일치, 집계/디듀프 안정성.

2) 공식 지원 시나리오(3개 + 1개 추가)

> 아래 3개는 "지원 범위를 명확히 하기 위한 공식 모드"이며, 이외 조합은 v4.2에서 지원하지 않는다.

시나리오 1) InsideSnapshot (Inside / No Trigger / External Observe) — 기본 주력 방식

상황

- CI/e2e에서 "테스트 시작~종료 구간"을 하나의 측정 윈도우로 삼는다.
- 컨트롤러 `/metrics` 가 클러스터 내부 Service로만 접근 가능하다.
- 프레임워크는 **클러스터 내부에 짧게 살아있는 curl pod**를 띄워 `/metrics` 를 scrape 한다.
- 결과는 `ArtifactsDir` 에 `sli-summary.v3...json` 로 저장된다. (파일명/충돌 처리 규칙은 v4.2 기준)

3-레이어 매핑

- Trigger: `NoTrigger` (세션 Start/End)
- Observe: `Inside/CurlPod` (Prom/OpenMetrics 텍스트 스크랩)
- Evaluate: v3 엔진 평가(요약 스키마는 slo.v3 유지)
- Write: ArtifactsDir 있을 때만 atomic write + (v4) 충돌 회피

설정 예시(필드 형태)

- **Common**
  - `Enabled: true`

    - `ArtifactsDir: "./_artifacts"`
    - `RunID: "ci-20260130-001"`
    - `Suite: "e2e"`
    - `TestCase: ""` (비워두면 자동으로 Ginkgo 현재 spec 이름 사용)
    - `Tags: {}` (자동으로 suite/test_case/namespace/run_id 등 추가됨)
    - `PresetSet: "Baseline"` (기본값)
    - `Format: "v4.2"` *(권장: summary의 config.format으로 기록)*
- **Method**
    - `Method: "InsideSnapshot"`
- **Trigger**
    - `Trigger.Mode: "None"` (고정)
- **Observe(Inside)**
    - `Target.Namespace: "my-operator-system"`
    - `Target.MetricsServiceName: "my-operator-controller-manager-metrics"`
    - `Target.MetricsPort: 8443` (기본값이면 생략 가능)
- **Auth (권장)**
    - `Auth.Mode: "ServiceAccountToken"` (기본값)
    - `Auth.ServiceAccountName: "my-operator-metrics-reader"`
- **FetchStrategy**
    - `FetchStrategy: "CurlPod"` (기본값)
    - `Fetch.CurlPod.Image: "curlimages/curl:latest"` (기본값)
    - `Fetch.CurlPod.ServiceURLFormat: "https://%s.%s.svc:8443/metrics"` (기본값)
- **Timeouts**
    - `Timeouts.Scrape: "2m"` (기본값)
    - `Timeouts.WaitPodDone: "5m"` (기본값)
    - `Timeouts.Logs: "2m"` (기본값)
- **Policy (v4.2에서 명시 권장)**
    - `Policy.BestEffortFetch: true` *(fetch 실패는 warning으로 강등)*

- `Policy.WriterFailure: "Fatal"` 또는 `"Warn"` *(프로젝트 기본값에 맞춰 문서화; v3/v4 기존 계약 유지가 목표면 현행과 동일하게)*

시나리오 2) InsideAnnotation (Inside / Annotation Trigger / External Observe) — CRD 설계 검증용 (v4.2 핵심 확장)

상황

- "테스트 윈도우"를 단순 start/end가 아니라, **특정 CR에 annotation을 넣는 순간부터**로 정의한다.
- 이 방식은 "실제 사용자 플로우에 가까운" 관측을 가능하게 해서 **CRD/컨트롤러 설계 검증**에 유리하다.
- v4.2에서는 **종료 정의(EndPolicy)를 명확히 표준화**하여 UX 흔들림을 줄인다.

3-레이어 매핑

- Trigger: `AnnotationTrigger` (시작: annotation write, 종료: EndPolicy)
- Observe: `Inside/CurlPod` (동일)
- Evaluate: 동일(v3 엔진 평가)
- Write: 동일

설정 예시(필드 형태)

- **Common**
  - `Enabled: true`
  - `ArtifactsDir: "./_artifacts"`
  - `RunID: "local-20260130-anno-01"`
  - `Suite: "e2e"`
  - `TestCase: ""` (자동 채움)
  - `PresetSet: "Baseline"` + (선택) `Specs: [ ...사용자 SLI 추가... ]`
  - `Format: "v4.2"`
- **Method**
  - `Method: "InsideAnnotation"`
- **Observe(Inside)**

    - `Target.Namespace: "my-operator-system"`
    - `Target.MetricsServiceName: "my-operator-controller-manager-metrics"`
    - `Target.MetricsPort: 8443`
- **Trigger (필수 최소셋: "어느 CR에 어떤 키로")**
    - `Trigger.Mode: "Annotation"`
    - `Trigger.ResourceRef`
        - `GVK: { group: "lab.heainseo.dev", version: "v1", kind: "SLOJob" }`
        - `Name: "slojob-sample-1"`
        - `Namespace: "default"` *(CR이 존재하는 namespace; metrics ns와 다를 수 있음)*
    - `Trigger.AnnotationKey: "test/start-time"`
- **Trigger Value (표준 동작)**
    - `Trigger.ValueMode: "RFC3339NanoNow"` (기본값)
    - `Trigger.Once: true` (기본값)
    - `Trigger.AutoRemoveKey: true` (기본값)
- **EndPolicy (v4.2 추가/강화 포인트)**
    - `EndPolicy.Mode: "StatusCondition"` *(권장 기본)*
        - `EndPolicy.Condition.Type: "Ready"`
        - `EndPolicy.Condition.Status: "True"`
        - `EndPolicy.Condition.SourceRef: <ResourceRef와 동일 또는 별도 지정 가능>`
    - 또는 `EndPolicy.Mode: "MetricDelta"` *(고급: 메트릭 기반 안정화/변화 감지)*
        - `EndPolicy.Metric.Key: "<canonical metric key>"`
        - `EndPolicy.Metric.Rule: "increases" | "stabilizes" | "reaches_threshold"`
        - `EndPolicy.Metric.Window: "30s"` *(관측 루프/안정화 판단 윈도우)*
    - 또는 `EndPolicy.Mode: "Timeout"` *(최후 보루/디버깅용)*
- **Auth/FetchStrategy/Timeouts**
    - InsideSnapshot과 동일 기본값(대부분 생략 가능)
    - 추가로:

    - `Timeouts.TriggerObserve: "10m"` *(EndPolicy 대기 포함)*

> 표준 종료 규칙(권장)
>
> - v4.2 기준으로는 UX 흔들림 방지를 위해 `StatusCondition`을 기본으로 두고, `MetricDelta`는 "실험/고급"으로 제공하는 구성이 안정적이다.

시나리오 3) OutsideSnapshot (Outside / No Trigger / External Observe) — 블랙박스 검증/디버깅

상황

- 컨트롤러를 "완전한 블랙박스"로 취급하고 **클러스터 밖에서** `/metrics`를 긁는다.
- 목적은 자동화 게이트라기보다:
  - "내 inside 계측이 시스템을 왜곡하지 않았나?"
  - "특정 환경에서 inside 방식이 막힐 때 디버깅"
- v4.2 원칙상 Outside는 **이미 접근 가능한 Endpoint가 준비된 경우만** 지원한다.

  (port-forward 자동화 같은 건 지원 범위에서 끊는다)

3-레이어 매핑

- Trigger: `NoTrigger` (테스트 Start/End)
- Observe: `Outside/Endpoint`
- Evaluate: 동일
- Write: 동일

설정 예시(필드 형태)

- **Common**
  - `Enabled: true`
  - `ArtifactsDir: "./_artifacts"`
  - `RunID: "debug-outside-20260130"`
  - `Suite: "debug"`
  - `TestCase: ""` (자동 채움)
  - `PresetSet: "Baseline"`
  - `Format: "v4.2"`

- **Method**
  - `Method: "OutsideSnapshot"`
- **Outside (필수: 접근 가능한 endpoint)**
  - `Outside.Endpoint: "http://127.0.0.1:18080/metrics"`
    - *(예: 사용자가 별도로 port-forward를 열어둔 상태)*
- **Outside Auth (필요 시)**
  - `Outside.Auth.Mode: "BearerToken"` 또는 `"None"`
  - `Outside.Auth.Token: "<redacted>"`
- **Outside TLS (필요 시)**
  - `Outside.TLS.InsecureSkipVerify: false` (기본값)
- **Timeouts**
  - `Outside.Timeouts.Scrape: "20s"` *(권장: 밖에서는 빠르게 실패시키는 편이 UX 좋음)*
- **Window**
  - `Window: "TestStartEnd"` *(NoTrigger이므로 기본)*

시나리오 4) - kube-burner 관련 내용 → 추가 survey 필요함.

참고

3) 비지원/비권장 조합(명시)

- **Outside + "경계 없음(주기 스크랩만)"**
  - 트리거도 없고, 테스트 start/end 같은 명확한 경계도 없이 "계속 긁기"는 SLI를 어느 구간에 귀속할지 모호해져서 v4.2에서는 공식 지원하지 않는다.
- **Inside + 프로덕션 코드 침투형 Internal Observe**
  - "오퍼레이터 프로덕션 코드에 계측 삽입 금지" 원칙 때문에, Internal observe는 필요 시에도 **테스트/하네스 내부에서만** 제한적으로 고려한다.

4) 산출물/네이밍/호환성(v4.2)

- Summary 스키마: 계속 `schemaVersion="slo.v3"` 유지

- `config.format` : `"v4.2"` 권장 (v3/v4 구분 및 분석 파이프라인에 유용)
- 파일 네이밍(기본):
  - `sli-summary.v3.<run_id>.<test_case>.json` *(sanitize 적용)*
  - v4 계열에서는 충돌 방지를 위해 `NextSummaryPath` 규칙을 사용(중복 실행/재시도 안전)
- `ArtifactsDir==""` 이면 write는 noop

5) OTel 전망에 대한 설계 포인트(간단 정리)

- v4.2의 기본 Observe는 Prometheus/OpenMetrics 스크랩이지만,
- 장기적으로 Observe 레이어를 플러그인화하면:
  - `/metrics` 스크랩(현재)과
  - OTLP(OTel) 기반 입력(미래)을

    같은 Evaluate 엔진으로 수렴시킬 수 있다.
- 따라서 v4.2 문서에서는 "입력 수집은 Observe 레이어 책임"을 원칙으로 고정한다.

6) 한 줄 요약(문서에 넣을 문장)

- **InsideSnapshot**: CI 기본값. "세션이 경계를 잡고, inside curl로 스크랩한다."
- **InsideAnnotation**: CRD 설계 검증. "annotation으로 시작을 정의하고, EndPolicy로 종료를 정의한다."
- **OutsideSnapshot**: 블랙박스 검증/디버깅. "이미 접근 가능한 endpoint가 있을 때만 사용한다."

7) 변경점 요약(v4.1 → v4.2에서 새로 추가된 것)

- **(추가) 3-레이어 모델(Trigger/Observe/Evaluate)로 방식 정의를 고정**
- **(강화) InsideAnnotation의 종료 정의를 EndPolicy로 명시**
  - `StatusCondition` / `MetricDelta` / `Timeout`
- **(정리) "외부 non-trigger 무의미"는 '경계 없음'일 때라는 조건을 명확화**
- **(미래 대비) Observe 레이어 플러그인화로 OTel(OTLP) 수용 여지 확보**